

AI Knowledge Hub

AWS EC2 Deployment Guide

Version 1.0 — Internal Reference

Scope: EC2 Web Application Infrastructure

Stack: Node.js / Docker / Amazon ECR / EC2 / ALB / DynamoDB / S3

Confidential — Internal Use Only

Contents

1. Executive Summary

This document provides complete, step-by-step instructions for deploying the AI Knowledge Hub web application on AWS EC2. It is written for a cloud engineer who has console access and will perform the setup manually through the AWS Management Console.

By the end of this guide, the following infrastructure will be in place:

- A private VPC with public and private subnets across two Availability Zones.
- An Application Load Balancer (ALB) in the public subnets, accepting HTTPS traffic.
- EC2 instances in private subnets, running the Node.js application inside Docker containers pulled from Amazon ECR.
- A VPC Endpoint granting the private EC2 instances private connectivity to DynamoDB and S3, with no internet traversal.
- An IAM Instance Role providing the application with automatic, credential-free access to AWS services.

Note: Complete the steps in the order they appear. Many steps depend on resources created in earlier steps. Do not skip sections.

2. Architecture Overview

The following table summarises the request flow from a user's browser to the database layer.

Step	Component	Description
1	User / Internet	Browser sends HTTPS request
2	Internet Gateway (IGW)	Entry point into the VPC
3	Application Load Balancer	Terminates TLS, distributes traffic across EC2 targets in the Auto Scaling Group
4	NAT Gateway	Allows EC2 instances in private subnets to initiate outbound connections (e.g. pull Docker images from ECR)
5	EC2 (private subnet)	Runs the Node.js application container. Receives traffic from the ALB only.
6	VPC Endpoint	Private route to DynamoDB and S3. Traffic never leaves the AWS network.
7	DynamoDB / S3	Paper metadata and audio files are read and written through the VPC Endpoint.

2.1 Why EC2 in Private Subnets?

Placing the application servers in private subnets means they have no public IP address and cannot be reached directly from the internet. All inbound traffic must pass through the ALB, which acts as the single controlled entry point. This reduces the attack surface significantly and is the accepted corporate standard for web application deployments.

2.2 Why a VPC Endpoint?

By default, AWS SDK calls to DynamoDB and S3 route over the public internet, even from within a VPC. EC2 instances in private subnets cannot reach the public internet without a NAT Gateway. A VPC Endpoint creates a private, internal route directly to these services, removing the dependency on NAT for database traffic. It is also free for Gateway-type endpoints (DynamoDB and S3).

3. Prerequisites

Before beginning, confirm the following are in place.

Requirement	Detail
AWS account with console access	Administrator or PowerUser IAM permissions required
DynamoDB tables already exist	papers table and subscriptions table must exist (created during Lambda pipeline setup)
S3 bucket already exists	The bucket storing audio files must exist
Docker installed locally	Required to build and push the application image
AWS CLI installed and configured locally	Required to authenticate Docker with ECR. Run: aws configure
Application code is production-ready	All code changes described in the Code Changes document must be applied before building the image

4. Step 1 — Create the ECR Repository

Amazon Elastic Container Registry (ECR) is a private Docker image registry hosted inside AWS. The EC2 instances will pull the application image from ECR at startup. This keeps the image private and ensures fast pulls with no egress cost within the same region.

4.1 Create the Repository

1. Open the AWS Management Console and navigate to Amazon ECR.
2. Click Get Started or Create repository.
3. **Repository name:** ai-knowledge-hub-backend
4. **Visibility:** Private
5. Leave all other settings as default and click Create repository.
6. After creation, click on the repository name and copy the URI shown at the top. It will look like:

```
<account-id>.dkr.ecr.<region>.amazonaws.com/ai-knowledge-hub-backend
```

4.2 Build and Push the Docker Image

Run the following commands from inside the backend/ directory of the project. Replace the placeholder values with your actual account ID and region.

```
# Step 1 – Authenticate Docker with ECR
aws ecr get-login-password --region <region> \
| docker login --username AWS \
--password-stdin <account-id>.dkr.ecr.<region>.amazonaws.com

# Step 2 – Build the image
docker build -t ai-knowledge-hub-backend .

# Step 3 – Tag the image for ECR
docker tag ai-knowledge-hub-backend:latest \
<account-id>.dkr.ecr.<region>.amazonaws.com/ai-knowledge-hub-backend:latest

# Step 4 – Push to ECR
docker push \
<account-id>.dkr.ecr.<region>.amazonaws.com/ai-knowledge-hub-backend:latest
```

Note: Every time the application code changes, repeat Step 4.2 to build and push a new image. The EC2 instances must then be refreshed (covered in Section 11).

5. Step 2 — Create the IAM Instance Role

The EC2 instances need permission to read from DynamoDB and S3 (and to pull images from ECR at startup). Instead of placing credentials anywhere in the code or container, an IAM Instance Role is attached to the EC2 instances. The AWS SDK detects this role automatically and uses it to sign all API requests. No keys, no secrets, no .env files in production.

5.1 Create the Role

7. Navigate to IAM in the AWS Console.
8. Click Roles in the left sidebar, then Create role.
9. **Trusted entity type:** AWS service
10. **Use case:** EC2 — select EC2 from the list and click Next.
11. On the permissions screen, attach the following managed policy:
 - **AmazonEC2ContainerRegistryReadOnly** — allows pulling images from ECR
12. Click Next, then set the following:
 - **Role name:** ai-knowledge-hub-ec2-role
13. Click Create role.

5.2 Add an Inline Policy for DynamoDB and S3

The managed policies do not cover DynamoDB or S3. Add a custom inline policy to grant exactly the permissions the application needs.

14. In IAM, click Roles and open ai-knowledge-hub-ec2-role.
15. Click Add permissions → Create inline policy.
16. Switch to the JSON editor and paste the following policy, replacing the placeholder values with your actual resource ARNs.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DynamoDBAccess",
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:PutItem",
```

```
        "dynamodb:Scan",
        "dynamodb:Query"
    ],
    "Resource": [
        "arn:aws:dynamodb:<region>:<account-id>:table/papers",
        "arn:aws:dynamodb:<region>:<account-id>:table/subscriptions"
    ]
},
{
    "Sid": "S3AudioAccess",
    "Effect": "Allow",
    "Action": [
        "s3:GetObject"
    ],
    "Resource": "arn:aws:s3:::<your-bucket-name>/*"
}
]
```

17. Click Next, name the policy ai-knowledge-hub-app-policy, then click Create policy.

Security note: This policy follows the principle of least privilege. The application only receives the exact actions it needs. No wildcard actions (*) are used on sensitive resources.

6. Step 3 — Create the VPC and Subnets

The VPC is the isolated private network inside AWS where all resources will live. The architecture requires two public subnets (for the ALB and NAT Gateway) and two private subnets (for the EC2 instances), spread across two Availability Zones for redundancy.

6.1 Create the VPC

18. Navigate to VPC in the AWS Console.
19. Click Your VPCs → Create VPC.
20. **Name tag:** ai-hub-vpc
21. **IPv4 CIDR block:** 10.0.0.0/16
22. Leave all other settings as default and click Create VPC.

6.2 Create Subnets

Create four subnets as described below. Navigate to Subnets → Create subnet and repeat for each row.

Name	Availability Zone	CIDR Block	Purpose
ai-hub-public-1	us-east-1a	10.0.1.0/24	ALB + NAT Gateway
ai-hub-public-2	us-east-1b	10.0.2.0/24	ALB (second AZ)
ai-hub-private-1	us-east-1a	10.0.3.0/24	EC2 instances
ai-hub-private-2	us-east-1b	10.0.4.0/24	EC2 instances (second AZ)

Important: Use the same Availability Zones throughout this guide. If your region uses different zone names (e.g. eu-west-1a), substitute accordingly. The zone names must be consistent across subnets, the ALB, and the Auto Scaling Group.

6.3 Create the Internet Gateway

23. Navigate to Internet Gateways → Create internet gateway.
24. **Name tag:** ai-hub-igw
25. Click Create internet gateway.
26. On the next screen, click Actions → Attach to VPC and select ai-hub-vpc.

6.4 Create the NAT Gateway

The NAT Gateway allows EC2 instances in private subnets to initiate outbound connections (for example, to pull Docker images from ECR on first boot). It must be placed in a public subnet.

27. Navigate to NAT Gateways → Create NAT gateway.
28. **Name:** ai-hub-nat
29. **Subnet:** ai-hub-public-1
30. **Connectivity type:** Public
31. Click Allocate Elastic IP, then Create NAT gateway.
32. Wait for the NAT Gateway status to become Available before continuing.

6.5 Configure Route Tables

Two route tables are needed: one for the public subnets (routing internet traffic through the IGW), and one for the private subnets (routing outbound traffic through the NAT Gateway).

Public route table

33. Navigate to Route Tables → Create route table.
34. **Name:** ai-hub-public-rt | **VPC:** ai-hub-vpc
35. Click Create route table.
36. Open the route table, go to the Routes tab, and click Edit routes → Add route.
37. **Destination:** 0.0.0.0/0 | **Target:** ai-hub-igw (Internet Gateway)
38. Click Save changes.
39. Go to the Subnet associations tab → Edit subnet associations and add ai-hub-public-1 and ai-hub-public-2.

Private route table

40. Create a second route table named ai-hub-private-rt in ai-hub-vpc.
41. Add a route: Destination 0.0.0.0/0, Target: ai-hub-nat (NAT Gateway).
42. Associate it with ai-hub-private-1 and ai-hub-private-2.

7. Step 4 — Create Security Groups

Security groups act as virtual firewalls. Two are required: one for the ALB and one for the EC2 instances. The EC2 security group is configured to accept traffic only from the ALB security group, not from the internet directly.

7.1 ALB Security Group

43. Navigate to EC2 → Security Groups → Create security group.

44. **Name:** ai-hub-alb-sg | **VPC:** ai-hub-vpc

45. Add the following inbound rules:

Type	Source	Purpose
HTTP (80)	0.0.0.0/0 (Anywhere IPv4)	Accept web traffic
HTTPS (443)	0.0.0.0/0 (Anywhere IPv4)	Accept secure web traffic

46. Leave outbound rules as default (All traffic allowed). Click Create security group.

7.2 EC2 Security Group

47. Create a second security group.

48. **Name:** ai-hub-ec2-sg | **VPC:** ai-hub-vpc

49. Add one inbound rule:

Type	Source	Purpose
Custom TCP / Port 3000	ai-hub-alb-sg (select the ALB security group by ID)	Allow traffic only from the ALB

50. Leave outbound rules as default. Click Create security group.

Security note: By setting the source of the EC2 inbound rule to the ALB security group (not an IP range), traffic is restricted to requests that have passed through the ALB. Direct access to port 3000 from the internet is blocked.

8. Step 5 — Create VPC Endpoints

VPC Endpoints create a private connection between the VPC and AWS services. Without them, the EC2 instances in private subnets would need to route all DynamoDB and S3 traffic through the NAT Gateway, which adds cost and sends traffic over the public AWS network. With Gateway endpoints, traffic stays entirely inside the AWS backbone and there is no additional charge.

8.1 *DynamoDB Endpoint*

51. Navigate to VPC → Endpoints → Create endpoint.
52. **Name:** ai-hub-dynamodb-endpoint
53. **Service category:** AWS services
54. In the search box, type DynamoDB and select com.amazonaws.<region>.dynamodb (type: Gateway).
55. **VPC:** ai-hub-vpc
56. Under Route tables, check the box next to ai-hub-private-rt.
57. Leave Policy as Full access. Click Create endpoint.

8.2 *S3 Endpoint*

58. Click Create endpoint again.
59. **Name:** ai-hub-s3-endpoint
60. Search for S3 and select com.amazonaws.<region>.s3 (type: Gateway).
61. **VPC:** ai-hub-vpc
62. Under Route tables, check ai-hub-private-rt.
63. Click Create endpoint.

Note: After creation, verify both endpoints appear as Available in the Endpoints list. No code changes are needed — the AWS SDK automatically routes through the endpoint once it is associated with the route table.

9. Step 6 — Create the EC2 Launch Template

A Launch Template defines the configuration used every time a new EC2 instance is started by the Auto Scaling Group. It specifies the AMI, instance type, security group, IAM role, and most importantly, a User Data script that runs on first boot to pull and start the Docker container.

9.1 Create the Launch Template

64. Navigate to EC2 → Launch Templates → Create launch template.
65. **Launch template name:** ai-hub-backend-lt
66. **Template version description:** Initial version

9.2 Configure the Template Settings

AMI (Amazon Machine Image):

- Click Browse more AMIs. Search for Amazon Linux 2023 and select the latest Amazon Linux 2023 AMI (64-bit x86). Note the AMI ID for your records.

Instance type:

- Select **t3.micro** for development/testing, or **t3.small** for production workloads.

Key pair:

- Select an existing key pair, or click Create new key pair to create one. Save the .pem file securely. This is only needed for emergency SSH access and should not be used for routine operations.

Network settings:

- Do **not** specify a subnet here. The Auto Scaling Group will place instances in the correct subnets.
- **Security groups:** Select ai-hub-ec2-sg
- **Auto-assign public IP:** Disable — instances are in private subnets and must not have public IPs

IAM instance profile:

- Under Advanced details → IAM instance profile, select **ai-knowledge-hub-ec2-role**

9.3 Write the User Data Script

The User Data script runs automatically on the first boot of every new EC2 instance. It installs Docker, authenticates with ECR, pulls the application image, and starts the container with the required environment variables. This is what replaces manually SSH-ing into a server.

Scroll down to Advanced details → User data and paste the following script. Replace all placeholder values before saving.

```
#!/bin/bash
set -e

# — Variables —————
REGION="<your-region>"                # e.g. us-east-1
ACCOUNT_ID="<your-account-id>"          # 12-digit AWS account ID
ECR_URI="${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com"
IMAGE="${ECR_URI}/ai-knowledge-hub-backend:latest"

# — Install Docker —————
yum update -y
yum install -y docker
systemctl enable docker
systemctl start docker

# — Authenticate with ECR —————
aws ecr get-login-password --region ${REGION} \
  | docker login --username AWS --password-stdin ${ECR_URI}

# — Pull the latest image —————
docker pull ${IMAGE}

# — Run the container —————
docker run -d \
  --name ai-hub-backend \
  --restart unless-stopped \
  -p 3000:3000 \
  -e PORT=3000 \
  -e AWS_REGION=${REGION} \
  -e PAPERS_TABLE=papers \
  -e SUBSCRIPTIONS_TABLE=subscriptions \
  -e S3_BUCKET=<your-s3-bucket-name> \
  ${IMAGE}
```

Important: The `--restart unless-stopped` flag ensures the container starts automatically if the EC2 instance reboots. No `.env` file is used. Environment variables are passed directly to the container via `-e` flags in the `docker run` command.

67. After pasting the script, click Create launch template.

10. Step 7 — Create the Load Balancer and Target Group

The Application Load Balancer (ALB) is the single public entry point for all web traffic. It distributes incoming requests across the healthy EC2 instances in the Auto Scaling Group. The Target Group defines how the ALB checks whether an EC2 instance is healthy before sending it traffic.

10.1 Create the Target Group

The Target Group must be created before the ALB, as the ALB will reference it during setup.

68. Navigate to EC2 → Target Groups → Create target group.
69. **Target type:** Instances
70. **Target group name:** ai-hub-backend-tg
71. **Protocol:** HTTP | **Port:** 3000
72. **VPC:** ai-hub-vpc
73. Under Health checks, configure the following:

Setting	Value
Health check protocol	HTTP
Health check path	/health
Healthy threshold	2 (two consecutive successes to mark healthy)
Unhealthy threshold	3 (three consecutive failures to mark unhealthy)
Timeout	5 seconds
Interval	30 seconds
Success codes	200

74. Click Next, skip adding targets for now (the Auto Scaling Group will register them automatically), and click Create target group.

10.2 Create the Application Load Balancer

75. Navigate to EC2 → Load Balancers → Create load balancer.
76. Choose Application Load Balancer and click Create.
77. **Load balancer name:** ai-hub-alb
78. **Scheme:** Internet-facing
79. **IP address type:** IPv4

80. Under Network mapping:

- **VPC:** ai-hub-vpc
- Select both Availability Zones and choose ai-hub-public-1 and ai-hub-public-2 respectively.

81. **Security groups:** Remove the default group. Add ai-hub-alb-sg only.

82. Under Listeners and routing, the default listener is HTTP:80. Set the default action to Forward to ai-hub-backend-tg.

83. Click Create load balancer.

84. After creation, copy the DNS name of the ALB (e.g. ai-hub-alb-123456789.us-east-1.elb.amazonaws.com). This is the URL used to access the application.

11. Step 8 — Create the Auto Scaling Group

The Auto Scaling Group (ASG) manages the fleet of EC2 instances. It uses the Launch Template to start new instances when needed and the Target Group to register them with the ALB. It also replaces unhealthy instances automatically.

85. Navigate to EC2 → Auto Scaling Groups → Create Auto Scaling group.

86. **Auto Scaling group name:** ai-hub-asg

87. **Launch template:** ai-hub-backend-1t (select the latest version)

88. Click Next.

89. Under Network:

- **VPC:** ai-hub-vpc
- **Availability Zones and subnets:** Select ai-hub-private-1 and ai-hub-private-2

90. Click Next.

91. Under Load balancing, select Attach to an existing load balancer.

92. Choose ai-hub-backend-tg from the list.

93. Enable EC2 instance health checks AND Elastic Load Balancing health checks. Set the health check grace period to 120 seconds (this gives the instance time to start the container before health checks begin).

94. Click Next.

95. Set the following capacity values:

Setting	Value
Desired capacity	2
Minimum capacity	1
Maximum capacity	4

96. Click Next through the remaining screens and click Create Auto Scaling group.

Note: With Desired capacity set to 2, two EC2 instances will launch immediately. Both will run the User Data script, pull the image from ECR, and register themselves with the Target Group. Once their /health checks pass, the ALB will begin distributing traffic to them.

12. Step 9 — Deploy the Frontend (Static Files)

The frontend is a React application that compiles to static files (HTML, CSS, JavaScript). These files are served by Nginx inside their own Docker container. The frontend container must know where to find the backend API at build time.

12.1 Build the Frontend Image

Run the following from inside the `frontend/` directory. The `VITE_API_BASE_URL` build argument tells the frontend where to send API requests. Since Nginx proxies `/api` to the backend container, this value is always `/api`.

```
# From the frontend/ directory

# Build the image
docker build \
  --build-arg VITE_API_BASE_URL=/api \
  -t ai-knowledge-hub-frontend .

# Create a frontend ECR repository first, then tag and push
docker tag ai-knowledge-hub-frontend:latest \
  <account-id>.dkr.ecr.<region>.amazonaws.com/ai-knowledge-hub-frontend:latest

docker push \
  <account-id>.dkr.ecr.<region>.amazonaws.com/ai-knowledge-hub-frontend:latest
```

12.2 Frontend Hosting Options

There are two standard approaches for serving the compiled frontend. Choose one based on your requirements.

Option	Approach	Best For
Option A	Run the frontend Docker container on the same EC2 instances alongside the backend, adding it to the User Data script	Simpler setup; recommended for this project
Option B	Upload the compiled <code>dist/</code> folder to S3 and serve it via CloudFront	Production scale; global CDN delivery

Option A — Add to User Data script

Add the following lines to the User Data script in the Launch Template (after the backend container is started) to also pull and run the frontend container.

```
# Pull and run the frontend container
FRONTEND_IMAGE="${ECR_URI}/ai-knowledge-hub-frontend:latest"
docker pull ${FRONTEND_IMAGE}

docker run -d \
  --name ai-hub-frontend \
  --restart unless-stopped \
  -p 80:80 \
  -e BACKEND_HOST=localhost \
  ${FRONTEND_IMAGE}
```

When both containers run on the same instance, the Nginx frontend proxies /api requests to the backend container on localhost:3000. The ALB listener should then target port 80 instead of 3000.

13. Step 10 — Verify the Deployment

After all steps are complete, verify the system is working in the following order.

13.1 Check EC2 Instances

97. Navigate to EC2 → Instances. Confirm two instances are in the Running state.

98. Click each instance and verify:

- IAM Instance Profile shows ai-knowledge-hub-ec2-role.
- Subnet is one of the private subnets (ai-hub-private-1 or ai-hub-private-2).
- No public IPv4 address is assigned.

13.2 Check Target Group Health

99. Navigate to EC2 → Target Groups → ai-hub-backend-tg.

100. Click the Targets tab.

101. Wait for both instances to show a Health status of Healthy. This confirms the /health endpoint is responding correctly and the ALB is routing traffic to them.

102. If an instance shows Unhealthy, check the EC2 System Log (right-click the instance → Monitor and troubleshoot → Get system log) for Docker startup errors.

13.3 Test the Application

103. Copy the ALB DNS name from EC2 → Load Balancers.

104. Open a browser and navigate to `http://<alb-dns-name>`.

105. The AI Knowledge Hub homepage should load.

106. Confirm the papers list loads (verifies DynamoDB connectivity through the VPC Endpoint).

107. Open a paper detail page and check that audio loads (verifies S3 connectivity and presigned URL generation).

108. Submit the subscription form with a test email (verifies the POST /subscriptions endpoint).

13.4 Test the Health Endpoint Directly

From your local machine, run:

```
curl http://<alb-dns-name>/health
```

Expected response:

```
{"status":"ok"}
```

14. Updating the Application

When the application code changes, the following procedure deploys the new version with zero downtime.

109. Make code changes and commit them.
110. From the backend/ directory, build and push a new image to ECR:

```
docker build -t ai-knowledge-hub-backend .
docker tag ai-knowledge-hub-backend:latest \
  <account-id>.dkr.ecr.<region>.amazonaws.com/ai-knowledge-hub-backend:latest
docker push \
  <account-id>.dkr.ecr.<region>.amazonaws.com/ai-knowledge-hub-backend:latest
```

111. Navigate to EC2 → Auto Scaling Groups → ai-hub-asg.
112. Click Instance refresh → Start instance refresh.
113. Set Minimum healthy percentage to 50%. This tells the ASG to replace instances one at a time, keeping at least one running throughout. Click Start.
114. The ASG will terminate old instances and launch new ones. New instances pull the latest image from ECR via the User Data script. Monitor progress in the Instance refresh tab.

Note: An instance refresh with 50% minimum healthy ensures there is always at least one instance serving traffic during the update. With two desired instances, one is replaced at a time, eliminating downtime.

15. Summary of Resources Created

The following table lists every AWS resource created by this guide.

Service	Resource Name	Purpose
ECR	ai-knowledge-hub-backend	Private Docker image registry
IAM	ai-knowledge-hub-ec2-role	Instance role granting app access to AWS services
IAM	ai-knowledge-hub-app-policy	Inline policy for DynamoDB and S3 access
VPC	ai-hub-vpc	Isolated private network
VPC	ai-hub-public-1 / 2	Public subnets for ALB and NAT Gateway
VPC	ai-hub-private-1 / 2	Private subnets for EC2 instances
VPC	ai-hub-igw	Internet Gateway
VPC	ai-hub-nat	NAT Gateway for outbound EC2 traffic
VPC	ai-hub-public-rt	Route table for public subnets
VPC	ai-hub-private-rt	Route table for private subnets
VPC	ai-hub-dynamodb-endpoint	Private route to DynamoDB
VPC	ai-hub-s3-endpoint	Private route to S3
EC2	ai-hub-alb-sg	Security group for the ALB
EC2	ai-hub-ec2-sg	Security group for EC2 instances
EC2	ai-hub-backend-lt	Launch Template for EC2 instances
EC2	ai-hub-backend-tg	Target Group with /health check
EC2	ai-hub-alb	Application Load Balancer
EC2	ai-hub-asg	Auto Scaling Group managing EC2 instances

End of Document